FOOD COURT EVENT MANAGEMENT SYSTEM

DOC-ID: FC-ARCH-2026-V1

System Architecture, CI/CD Pipeline & Workflow Handbook

Official technical handbook for dual-environment deployment, automated CI/CD staging, and live production management.

PRODUCTION: LIVE | TEST: ACTIVE (:8080) | HOST: AWS EC2 (13.127.45.22) | STACK: NEXT.JS 16 + FASTAPI + POSTGRESQL 18

01 Dual-Environment Quick Reference

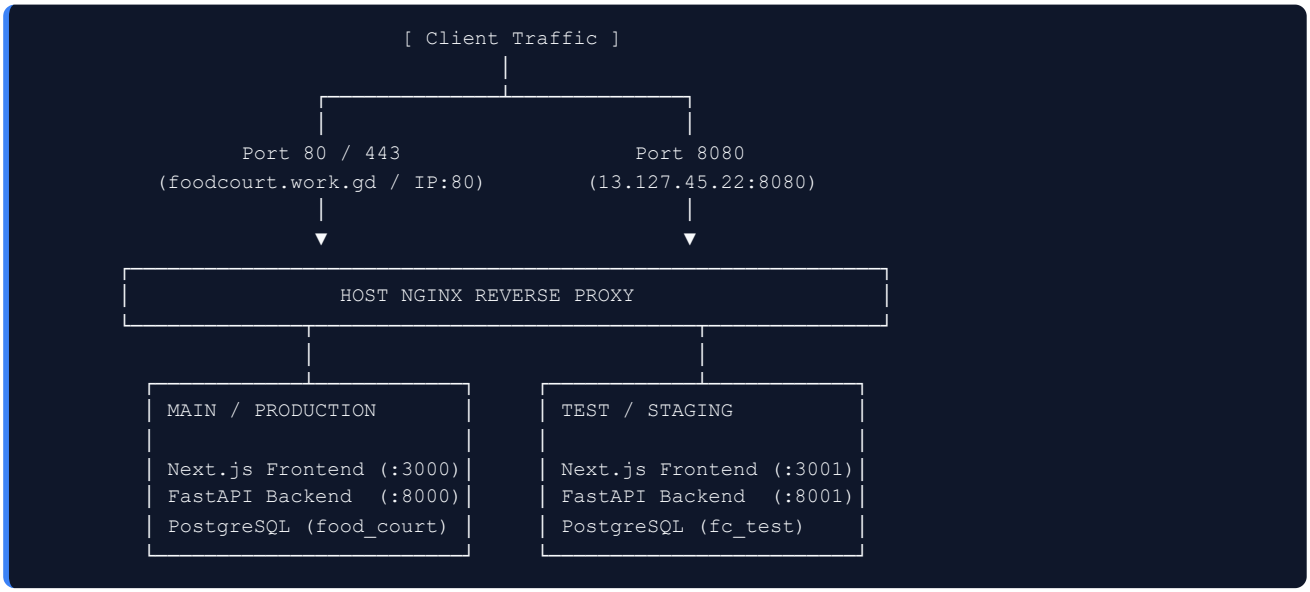
The host server operates two entirely separated installations side-by-side using Docker Compose and Nginx. This architecture allows developers to test features without customer risk.

Environment	Access URL	Ports (Front / Back)	Server Directory	Database	Git Branch
Main (Production)	`http://13.127.45.22` `https://foodcourt.work.gd`	Frontend: 3000 Backend: 8000	`/home/ubuntu/food-court-app`	`food_court`	`main`
Test (Staging)	`http://13.127.45.22:8080` `http://test.foodcourt.work.gd`	Frontend: 3001 Backend: 8001	`/home/ubuntu/food-court-test`	`food_court_test`	`test`

Database Isolation Guarantee: The test database is `food_court_test` and production is `food_court`. Any inserts, updates, or deletes conducted on the test site are strictly isolated from production data.

02 High-Level System Architecture

Nginx acts as the single gateway on the host, decrypting SSL and routing traffic based on the incoming port and domain header.



03 Nginx Reverse Proxy & Routing Rules

Configuration located at `/etc/nginx/sites-available/foodcourt`:

Production Blocks (Port 80 & 443)

Routes domain `foodcourt.work.gd` and IP port 80 to port 3000 (UI) and port 8000 (/api/py/ FastAPI).

Test Blocks (Port 8080 & Subdomain)

Routes port 8080 and `test.foodcourt.work.gd` to port 3001 (UI) and port 8001 (/api/py/ FastAPI).

FastAPI Swagger Documentation Routing:
Swagger UI requires `/openapi.json`. An explicit Nginx rule proxies `location = /openapi.json` directly to FastAPI, preventing Next.js from capturing it and throwing 404 errors.

04 Git Branching & Promotion Strategy

Developers work strictly according to the staging-first workflow:

1. **Develop on `test` Branch**
   All feature changes and fixes are developed locally on `test` and pushed to `origin test`.

2. **Verify on Staging Server**
   Test the staging environment at `http://13.127.45.22:8080`. Check UI, database actions, and API docs.

3. **Promote & Merge to `main`**
   Once verified, merge `test` into `main`, push to GitHub, and deploy to live production.

05 GitHub Actions CI/CD Pipeline

Configured in `.github/workflows/ci.yml`. Triggers automatically on push events to either `test` or `main`.

Job Name	Execution Condition	Primary Responsibility
backend-ci	Every push & PR	PostgreSQL 18 service, Python syntax compilation, and route registration smoke tests.
frontend-ci	Every push & PR	Node.js 20 environment, TypeScript strict check (`npx tsc --noEmit`), and Next.js production build.
docker-validation	After CI passes	Builds both Dockerfiles via Buildx and validates multi-service `docker compose config`.
deploy-test	Push to `test`	Automated SSH connection to `/home/ubuntu/food-court-test` and container rebuild.
deploy-prod	Push to `main`	Automated SSH connection to `/home/ubuntu/food-court-app` and zero-downtime container release.

06 Day-to-Day Deployment Commands

Phase 1: Deploying to Test Server

```
# 1. Switch to test branch and commit your work
git checkout test
git add .
git commit -m "feat: your new feature"
git push origin test

# 2. Deploy directly to the test server
ssh -i C:\Users\profe\.ssh\food-court.pem ubuntu@13.127.45.22 \
  "cd /home/ubuntu/food-court-test && git pull origin test && docker compose -p foodcourt-test up -d --b
```

Phase 2: Promoting Test to Live Production

```
# 1. Merge verified test changes into main
git checkout main
git merge test
git push origin main
git checkout test

# 2. Deploy to the live production server
ssh -i C:\Users\profe\.ssh\food-court.pem ubuntu@13.127.45.22 \
  "cd /home/ubuntu/food-court-app && git pull origin main && docker compose up -d --build"
```

07 One-Click Windows Batch Scripts

Included in the repository root for single-command automation:

deploy_test.bat

Automates git commit, push to `test`, and remote server update for port 8080.

```
.\deploy_test.bat "commit message"
```

deploy_prod.bat

Automates merge from `test` into `main`, push, and remote production server update.

```
.\deploy_prod.bat
```

08 Server Maintenance & AWS Security Rules

Required AWS EC2 Security Group Inbound Rules:

Protocol / Port	Port Range	Source	Purpose
SSH	22	Anywhere (0.0.0.0/0)	Secure terminal administration & GitHub Actions CD
HTTP	80	Anywhere (0.0.0.0/0)	Production web application & SSL ACME challenges
HTTPS	443	Anywhere (0.0.0.0/0)	Production encrypted SSL traffic
Custom TCP	8080	Anywhere (0.0.0.0/0)	Test / Staging application web traffic

Server Health & Memory Check:

```
# Check active containers
ssh -i C:\Users\profe\.ssh\food-court.pem ubuntu@13.127.45.22 "docker ps"

# Check server memory & swap
ssh -i C:\Users\profe\.ssh\food-court.pem ubuntu@13.127.45.22 "free -m"

# Free unused Docker build cache
ssh -i C:\Users\profe\.ssh\food-court.pem ubuntu@13.127.45.22 "docker system prune -f"
```